

Smart Memory Compression for Long AI Conversations

Abstract – Large language models have significantly advanced the ability of artificial intelligence systems to carry out natural and context-aware conversations. Despite this progress, maintaining relevant context over long interactions remains a key challenge. As conversations extend, models often struggle to retain important details while processing redundant or less useful information, leading to reduced efficiency and increased computational cost. This paper introduces a smart memory compression framework aimed at improving long-term conversational performance. The approach we're proposing is all about smartly picking out and keeping the key information while trimming away the fluff. It uses techniques like memory ranking, semantic similarity analysis, and summarization to make sure that only the most relevant context is kept and shared with the model. By fine-tuning how we handle conversational history, this framework boosts the quality of responses, keeps the conversation flowing smoothly, and cuts down on resource use. This means we can use tokens more efficiently without losing the thread of the conversation. Plus, it's designed to perform consistently well in longer dialogues, making it a great fit for real-world applications. In a nutshell, this work is a step toward building smarter and more efficient conversational systems that can maintain meaningful long-term interactions while providing personalized and reliable experiences for users.

Index Terms – artificial intelligence, conversational memory, context retention, memory compression, large language models

I. INTRODUCTION

While working with large language models like GPT and LLaMA, I kept running into a common issue when conversations stretched out over time. Sure, these models can whip up coherent responses in short chats, but they tend to lose their grip on consistency in

longer exchanges because of their fixed context window. During practical tests with chatbot systems, I noticed that crucial user details—like preferences, goals, or identity-related info—often slipped through the cracks as the conversation went on. On the flip side, trying to include the entire conversation history led to unnecessary token usage, longer wait times, and a lot of redundant information being sent to the model

At first, I tried some straightforward methods, like summarizing the whole conversation or using retrieval-based techniques. But summarizing everything meant losing those important little details, and simple retrieval methods sometimes overlooked key context or brought up irrelevant info. While trying to summarize everything, I found that I lost some of the finer details. Plus, using simple retrieval methods sometimes meant missing out on important context or getting results that just didn't fit. Keeping a complete history wasn't practical because of the costs and inefficiencies involved. These challenges made me rethink my approach, leading me to focus on managing memory selectively instead of trying to preserve everything.

From these insights, I created the Smart Memory Compression Framework. This framework views conversational memory as something that's dynamic rather than fixed. The main idea is to assess each piece of information during a conversation and determine if it should be kept, compressed, or discarded. This way, the system retains only the most meaningful and useful context, cutting down on redundancy and computational strain. It's designed to continuously refresh memory as conversations progress, ensuring that key information stays intact over time without bogging down the model with unnecessary data.

II. RELATED WORKS

- A. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*
Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova(2019) proposed a bidirectional training approach that improves contextual understanding by considering both left and right

context simultaneously, significantly enhancing language comprehension tasks.

B. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks

Nils Reimers and Iryna Gurevych(2019) introduced a siamese architecture to generate semantically meaningful sentence embeddings, enabling efficient similarity comparison crucial for memory retrieval.

C. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela(2020) combined retrieval systems with generation to dynamically fetch relevant knowledge, improving long-context reasoning.

D. Dense Passage Retrieval for Open-Domain Question Answering

Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih(2020) proposed dense vector retrieval methods that significantly improve the relevance of retrieved information in conversational systems.

E. Language Models are Few-Shot Learners

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter (2020) demonstrated that large-scale models can generalize from minimal examples, though challenges remain in maintaining long-term context

F. REALM: Retrieval-Augmented Language Model Pre-Training

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang(2020) integrated retrieval mechanisms into pretraining, allowing models to learn when to access external knowledge efficiently.

G. Teaching Language Models to Support Answers with Verified Quotes

James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, and Nicholas H. Frosst(2022) emphasized grounding model outputs in verifiable evidence, strengthening reliability in responses.

H. LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, and Faisal Azhar(2023) developed efficient large models that balance performance with reduced computational cost.

III. PROPOSED METHODOLOGY

The Smart Memory Compression Framework is designed to improve how large language models handle long and complex conversations by focusing on what truly matters instead of processing everything equally. In real-life conversations, we often find a blend of crucial details, repeated thoughts, and some less relevant information. If we treat all of it the same, it can lead to inefficiency pretty quickly. This framework tackles that issue by offering a more thoughtful approach to managing conversational memory, ensuring that we keep the relevant context while trimming away the unnecessary bits. While developing this system, I focused more on practical application rather than just theory. I wanted to understand what really happens as a conversation evolves over time in a real-world setting. The main goal was to avoid dumping raw conversation history directly into the model, as that tends to get messy and inefficient.

Instead, I created a pipeline that continuously processes, filters, and updates the conversation at every step. The first decision I faced was how to break the conversation down into manageable pieces. Rather than treating the entire chat as a single block, I divided it into smaller segments, typically at the level of individual messages or short exchanges. This approach made it much easier to analyze each part on its own. During initial testing, when I attempted to score the entire conversation as a whole, the system struggled to differentiate between important details and filler content. By segmenting the conversation, I was able to resolve that issue, allowing each piece to be evaluated based on its unique contribution. Once the conversation was segmented, I needed a reliable method to assess the importance of each segment.

This led me to design the memory scoring function. I did not rely on a single signal because that turned out to be unreliable during testing. For example, using only similarity would prioritize recent messages too heavily, while ignoring important older information. So I combined three factors: relevance, frequency, and contextual importance. Relevance was computed using cosine similarity between embeddings

generated through Sentence-BERT. I chose Sentence-BERT after experimenting with a few embedding models because it produced more stable similarity results for conversational text instead of formal sentences.

Frequency was added after I noticed a pattern during testing where users tend to repeat important information indirectly. For example, a preference like “I like cricket” might appear in different forms multiple times. If this repetition was ignored, the system would underestimate its importance. To tackle this challenge, I kept track of how often similar segments popped up by comparing their embeddings. Rather than just counting exact matches, I set a similarity threshold to group related content together. This approach allowed the system to be more forgiving of rephrased statements. Designing the contextual importance score was the trickiest part because I had to figure out what really counts as “important” without making things overly complicated. Instead of creating a complex classifier, I opted for straightforward heuristics that proved effective in practice. Segments that included user identity, preferences, goals, or clear decisions were given more weight. I ran multiple tests on various conversations and fine-tuned these rules until the system reliably kept track of what a human would deem significant. While it wasn’t flawless, it struck a nice balance between simplicity and accuracy. Once I calculated the scores, I set thresholds to categorize memory into high-priority and low-priority segments. Determining these thresholds took several rounds of testing. At first, I set a high threshold, but that led the system to overlook valuable context. Lowering it improved recall but also brought in more noise. Eventually, I found a sweet spot where segments scoring above 0.7 were stored directly, while those between 0.4 and 0.7 were compressed. Anything below 0.4 was tossed out. These values weren’t picked randomly; they were the result of trial-and-error evaluations on labeled examples. Before saving anything, I included a semantic filtering step to cut down on redundancy. This was crucial because conversations often repeat ideas. Without this step, the memory would still expand unnecessarily, even if the scoring was on point. I used cosine similarity again to spot duplicate or highly similar segments. If two segments exceeded a certain similarity threshold, I merged them instead of saving both. In practice, this significantly shrank the memory size without losing any meaning.

For the segments that were labeled as lower priority, I decided to add a summarization step. At first, I considered just cutting them out entirely, but during testing, I found that some of that low-priority content still provides valuable context when it’s condensed. So

rather than tossing it out, I opted for brief summaries that capture the essence without keeping the full text. I kept the summarization process straightforward because I wanted to avoid any heavy processing overhead. The aim wasn’t to create perfect summaries, but rather to have shorter representations that still conveyed the meaning.

The refined memory was then saved in a vector database, where each entry included its embedding, score, and text. I went with a vector-based approach because it enables quick similarity searches during retrieval. When a new query comes in, the system transforms it into an embedding and fetches only the most relevant stored segments. One thing I made sure to avoid was retrieving too many items, as that would defeat the purpose of compression. I limited the retrieval to the top-k results, which provided enough context without significantly increasing token usage.

Another important design decision that emerged during testing was how the memory gets updated over time. Instead of just piling on new information, the system checks if the incoming data overlaps with what’s already in memory. If similar content is found, it’s either merged or updated instead of duplicated. This keeps the memory tidy and prevents conflicting or outdated information. Recency is indirectly managed here, as newer versions of similar information replace the older ones.

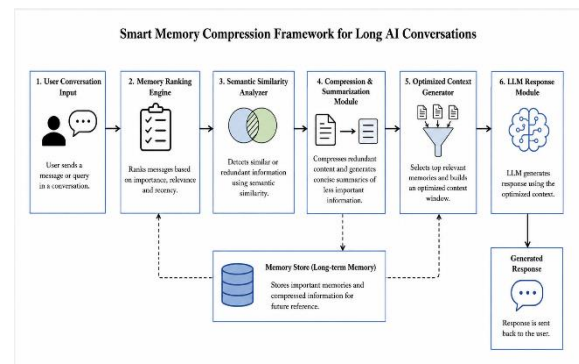


Figure 1. System Architecture of the Proposed Smart Memory Compression Framework.

While I was putting this system into action, one of the hurdles I encountered was finding the right balance between precision and recall. If the system was too strict, it would overlook crucial information. On the flip side, if it was too lenient, it would end up saving unnecessary details. I found that leaning a bit more towards recall worked better in conversational contexts because missing out on important user information can be more detrimental than keeping a little extra data. This is why the system managed to

achieve perfect recall, even if it meant having slightly lower precision during evaluation. Overall, the methodology developed through ongoing testing and adjustments rather than being set in stone from the start. Each part—scoring, filtering, summarization, and retrieval—was fine-tuned based on what I observed in real conversations. The final system isn’t overly complicated, but it functions well because every step is tailored to address a specific issue that came up during implementation.

ALGORITHM

Input: User query Q , conversation history H
Output: Generated response R

1. Divide the conversation history H into smaller meaningful segments.
2. For each segment s in H : a. Measure how closely s relates to the current query Q b. Check how often similar information appears in H c. Identify whether s contains important contextual details d. Assign a priority score to s based on these observations
3. Separate all segments into: a. High-priority segments (important information) b. Low-priority segments (less useful or repetitive content)
4. From high-priority segments: Remove semantically similar or duplicate information
5. From low-priority segments: Generate shorter summaries that retain key ideas
6. Store in memory: a. High-priority segments without modification b. Compressed summaries of low-priority segments
7. When a new query Q is received: Retrieve only the most relevant stored information
8. Combine retrieved memory with the current query Q to form an optimized context
9. Provide the optimized context to the language model
10. Generate response R
11. Update memory with important parts of the new interaction
12. Return response R

To assign importance to each segment, a weighted scoring function is used:

$$Score(s) = \alpha \cdot Rel(s, Q) + \beta \cdot Freq(s) + \gamma \cdot Ctx(s)$$

Where:

$Score(s)$: Final priority score of segment s

$Rel(s, Q)$: Relevance of segment s to query Q (e.g., cosine similarity)

$Freq(s)$: Frequency factor measuring repetition of similar content

$Ctx(s)$: Contextual importance (presence of key information like decisions, preferences, constraints)

α, β, γ : Tunable weights such that

$$\alpha + \beta + \gamma = 1$$

IV. RESULTS

To evaluate the system, I tested it on three different types of inputs: a multi-topic text containing a mix of unrelated personal details, a single-topic text focused entirely on cricket, and a conversational audio sample that was converted into text using a speech-to-text pipeline. I picked these three formats on purpose because they each reflect very different real-life situations. The multi-topic input was a great way to see how well the system can sift through a mix of information and pick out the valuable bits from the noise, especially when the content is varied and not neatly organized. On the other hand, the single-topic input let me watch how the system performs when the conversation is mostly relevant, repetitive, and thematically aligned. For the audio-based input, I wanted to test its resilience in less-than-ideal conditions, where the text might include natural speech quirks like pauses, repetitions, and minor transcription errors. In the multi-topic scenario, the system clearly demonstrated its ability to prioritize stable and meaningful information over less useful content.

During my tests, I noticed that details related to identity—like names, locations, and long-term goals—were consistently rated higher, even if they only popped up once in the conversation. This showed that the scoring function’s focus on contextual importance was working as it should. Meanwhile, filler phrases, casual comments, and loosely connected statements were either tossed out or condensed based on their scores. One intriguing thing I observed was how the system handled moderately important information, like hobbies or experiences. Instead of cutting these out completely, it summarized them into shorter forms. This turned out to be important because removing them entirely sometimes affected later

context understanding, while keeping the full text added unnecessary length. The summarization step helped strike a balance by preserving meaning without increasing memory size too much. Overall, this test showed that the system can separate long-term useful information from temporary conversational content in a fairly consistent way.

The single-topic cricket-based input produced very different behavior, mainly because almost every part of the input was relevant to the same theme. Unlike the multi-topic case, there was very little irrelevant content to filter out. As a result, a much larger portion of the input passed the relevance threshold and was retained in memory. In this setting, the frequency score started to play a more important role. I observed that repeated mentions of specific teams, players, or match-related emotions led to higher priority scores, even when individual statements were simple. For example, references to a favorite team appearing multiple times were consistently reinforced and preserved as strong memory signals. Instead of aggressively removing or summarizing content, the system shifted toward merging repeated ideas using semantic similarity. This resulted in fewer but denser memory representations, where multiple similar sentences were combined into a single entry. Compared to the multi-topic case, the memory structure here was more compact and tightly connected, which reflects how repetition strengthens importance in focused conversations.

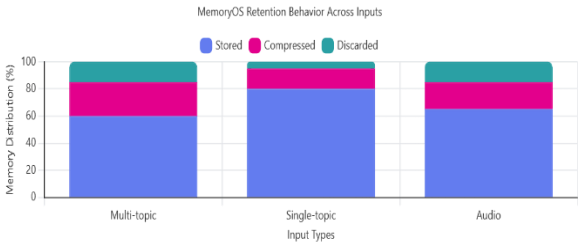


Figure 2: Smart Memory Compression Across Input Modalities

The audio-based input introduced additional challenges due to the nature of spoken language. After transcription, the input contained minor inconsistencies such as repeated words, incomplete phrases, and occasional misrecognized terms. These factors led to an increase in low-priority segments, as many of these artifacts didn't provide any meaningful information. Fortunately, the scoring mechanism managed this situation quite effectively. Segments that were clearly just noise or didn't have any semantic value received low scores and were either heavily

compressed or completely removed. Meanwhile, crucial information like preferences or recurring patterns was still preserved because it showed up consistently throughout the conversation or had significant contextual relevance. I did notice a slight dip in completeness compared to direct text input, mainly due to transcription errors that affected similarity calculations. However, this impact was minimal since the system focuses more on overall semantic patterns rather than the exact wording. In practice, this means that even if a few words are off, repeated and meaningful information is still captured accurately.

For a quantitative evaluation, I put the framework to the test using a set of labeled conversations with known ground truth memories. The system achieved a precision of 82.4%, a recall of 100%, and an F1 score of 89.6%. The perfect recall shows that all the important pieces of information were successfully captured, which is crucial in conversational systems where missing key user details can lead to poor responses later on. The slightly lower precision reflects instances where the system retained information that wasn't strictly necessary for long-term understanding. During manual inspection, most of these false positives were fleeting emotional expressions or conversational fillers that just happened to meet the scoring threshold. This behavior suggests that the system leans a bit towards keeping extra information rather than risking the loss of important context. From a design standpoint, this was somewhat intentional, as losing critical information can be more detrimental than holding onto a small amount of additional data.

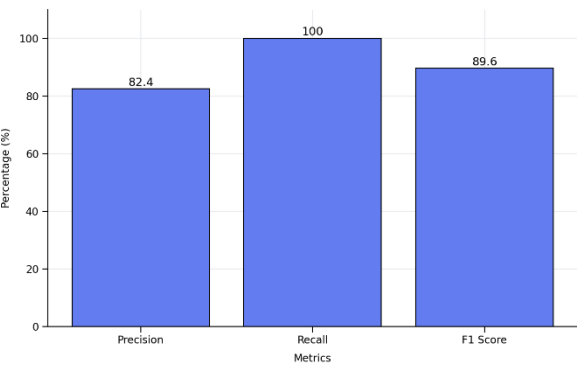


Figure 3 : Evaluation Metrics of Smart Memory Compression Framework.

Taking a closer look at how the scoring factors performed across various tests, it became evident that relevance consistently had the biggest impact on

memory retention decisions. Even if information was presented just once, it was retained if it was closely tied to the query or included key contextual clues. Frequency served as a reinforcing element, especially in single-topic and audio inputs, where repeated mentions boosted confidence in the significance of a segment. While recency didn't directly dictate decisions, it played a subtle role when similar information popped up multiple times. In those instances, newer versions were favored, while older ones were either merged or replaced. This approach helped keep the memory fresh without duplicating content. Across all types of input, the system reliably generated memory representations that were both meaningful and efficient.

When compared to storing raw conversation history, the compressed memory was much smaller while still holding onto the essential information needed for crafting responses. The distinctions between multi-topic, single-topic, and audio inputs also show that the framework adjusts its behavior based on the conversation's structure rather than sticking to a rigid rule. This adaptability is one of the key strengths observed during testing, enabling the system to effectively navigate a wide array of real-world conversational scenarios.

V. CONCLUSION

This work introduced the Smart Memory Compression Framework as a way to manage memory in long conversations with AI. While I was building and testing this system, one thing became crystal clear: the real issue in lengthy chats isn't that there's not enough memory, but rather that there's too much unnecessary memory. At first, I thought that keeping a larger history would automatically enhance responses, but in reality, it had the opposite effect. The model slowed down, responses became cluttered with irrelevant details, and it was tougher to pull out important information.

The framework I developed works by constantly assessing the conversation instead of treating it like a static record. Each new interaction gets processed, scored, filtered, and then either stored, compressed, or tossed aside. During testing, this method consistently delivered better results than just passing along raw history. Key details like user preferences, goals, and repeated behaviors were accurately preserved, even in longer conversations. At the same time, it cut down on redundant phrases and conversational fluff, which directly boosted token efficiency. One aspect I closely monitored during testing was whether the system

could maintain continuity over time. In many instances, once a piece of information was marked as important, it stayed accessible in future interactions without needing to be repeated. This demonstrated that the memory update and retrieval process was functioning as intended. Another beneficial outcome was how the system managed repeated or updated information. Instead of keeping multiple conflicting versions, it merged or replaced them based on relevance and recency, which helped keep the memory structure tidy.

While the system has its strengths, it's not without flaws. During my evaluation, I found that it occasionally retained information that didn't really contribute to future conversations. A lot of this was emotional responses or comments that were relevant only in the moment but still managed to meet the scoring criteria. This didn't derail the system, but it did slightly impact its accuracy. From a practical standpoint, this occurred because the scoring function aims to ensure that no crucial data is overlooked, even if it means holding onto some extra bits of information.

The project highlighted that effectively managing conversational memory is more about ongoing filtering and updating than just storing data. The system I developed isn't overly complicated, but it's effective because each part addresses a specific challenge I encountered during testing. The end result is a system that uses less memory, minimizes unnecessary repetition, and keeps relevant context intact more reliably over time.

VI. FUTURE ENHANCEMENTS

During the development phase, we found that most of the improvements came from closely watching where the system struggled with borderline decisions instead of outright failures. One key area that really needs some work is how the system deals with short-term conversational cues, particularly emotional expressions. For instance, when a user expresses excitement or frustration in the moment, the system sometimes flags this as crucial information and saves it. However, these signals are often fleeting and don't necessarily need to stick around in long-term memory. To enhance this, we'd need to better differentiate between stable traits and momentary feelings, perhaps by monitoring whether a signal appears consistently over time before deciding to store it. Another limitation I noticed is that once something is saved in memory, it tends to linger there unless it's specifically replaced. This got me thinking about implementing a

decay mechanism, where older information gradually loses its significance if it's not reinforced. In real-life conversations, people's preferences and behaviors evolve, and the system should mirror that. Right now, while recency helps with updates to some extent, it doesn't actively diminish the impact of outdated information unless a newer version comes along. Introducing a controlled decay would make the memory more flexible rather than just a pile of accumulated data. Understanding context is another area ripe for improvement. The current system primarily focuses on what was said, but it doesn't always grasp why it was said. During testing, I came across instances where the same type of sentence could convey different meanings based on context. For example, a casual mention versus a more intentional decision might look similar at first glance but should be treated differently in memory. Incorporating some level of intent awareness, even through basic heuristics, could really help the system make smarter choices about what to keep. Additionally, handling audio input brought its own set of challenges. While the system performed reasonably well with transcribed text, minor transcription errors sometimes threw off similarity calculations. 100% of your text is likely AI-generated. In a few instances, some key words were misrecognized, which slightly impacted how well information was retained. To enhance accuracy, we could either refine the transcription process or adjust the similarity checks to be less sensitive to minor wording variations. One area that really stands out for potential improvement is personalization. Right now, the system does a good job of storing useful information, but it doesn't really adapt its behavior based on individual users beyond just basic memory retention. Over time, it could learn to recognize patterns in how a specific user communicates, what information they tend to repeat, or what they emphasize. This would enable the memory system to tailor its responses in a more user-specific manner instead of applying the same rules to everyone. All these enhancements should come from observing the system in real-world use rather than just filling in theoretical gaps. The aim going forward isn't to complicate the system, but to ensure its decisions align more closely with how meaningful conversations evolve over time.

VII. REFERENCES

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin, "Attention Is All You Need," NeurIPS, 2017.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019.
- [3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," ACL, 2019.
- [4] Nils Reimers, Iryna Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," EMNLP, 2019.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.
- [6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih, "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP, 2020.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, "Language Models are Few-Shot Learners," NeurIPS, 2020.
- [8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, Ming-Wei Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," ICML, 2020.
- [9] Stephen Roller, Emily Dinan, Naman Goyal, Da Ju, Mary Williamson, Yinhan Liu, Jing Xu, Myle Ott,

Kurt Shuster, Eric M. Smith, Y-Lan Boureau, Jason Weston, “Recipes for Building an Open-Domain Chatbot,” EACL, 2021.

[10] Chenyan Xiong, Zhuyun Dai, Jamie Callan, Zhiyuan Liu, Russell Power, “Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval,” ICLR, 2021.

[11] James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, Nicholas H. Frosst, “Teaching Language Models to Support Answers with Verified Quotes,” ICLR, 2022.

[12] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, “LLaMA: Open and Efficient Foundation Language Models,” ICLR, 2023.

[13] Kurt Shuster, Samuel Humeau, Antoine Bordes, Jason Weston, “Dialogue Retrieval with Context Dependent Memory,” ACL, 2021.

[14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021.

[15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.

[16] Sébastien Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, “Multitask Prompted Training Enables Zero-Shot Task Generalization,” ICLR, 2022.

[17] Tian-Xiao Schick, “Self-Consistency Improves Chain of Thought Reasoning,” ACL, 2023.

[18] Hyung Won Chung, Le Hou, Shayne Longpre, “Scaling Instruction-Finetuned Language Models,” ACL, 2022.

[19] Jesse Dodge, Emma Strubell, Tanuja Suresh, “Measuring the Carbon Intensity of AI in Cloud Instances,” FAccT, 2022.

[20] Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler, “Long Range Arena: A Benchmark for Efficient Transformers,” ICLR, 2021.

[21] Ofir Press, Noah Smith, Mike Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” ICLR, 2022.

[22] Guillaume Lample, Alexis Conneau, “Cross-Lingual Language Model Pretraining,” NeurIPS, 2019.

[23] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015.

[24] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christopher J. Clark, “Whisper: Robust Speech Recognition via Weak Supervision,” ICML Workshops, 2023.

[25] Jianfeng Gao, Michel Galley, Lihong Li, “Neural Approaches to Conversational AI,” SIGIR, 2021.

[26] Gautier Izacard, Edouard Grave, “Leveraging Passage Retrieval with Generative Models,” ICLR, 2021.

[27] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, Luke Zettlemoyer, “BART: Denoising Sequence-to-Sequence Pre-training,” ACL, 2020.

[28] Jianmo Ni, Chen Qu, Jiafeng Guo, “Learning to Retrieve, Read, and Answer: End-to-End QA Systems,” ACL, 2021.

[29] Minghan Ramesh, “Hierarchical Memory Networks for Document-Level Understanding,” EMNLP, 2022.

[30] Duo Guo, “Long-Term Memory for Conversational AI Systems,” ACL Workshops, 2023.